

Memory Management

1. Types of Memories and the Need for Memory Management

1.1 Types of Memories

1. Primary Memory (RAM):

- Volatile memory used for storing currently executing processes and data.
- Fast but limited in size.
- Example: DDR4 RAM.

2. Secondary Memory:

- Non-volatile, used for long-term storage.
- Slower than primary memory.
- Example: Hard Drives (HDDs) and Solid State Drives (SSDs).

3. Cache Memory:

- High-speed memory located close to the CPU.
- Stores frequently accessed data.
- Example: L1, L2, L3 cache.

4. Virtual Memory:

- Logical extension of RAM on the hard disk.
 - Allows execution of programs larger than physical memory.
-

1.2 Need for Memory Management

- Efficient allocation of memory to multiple processes.
 - Prevents memory wastage.
 - Allows concurrent execution by ensuring isolation between processes.
 - Handles fragmentation and optimizes performance.
-

2. Continuous and Dynamic Memory Allocation

2.1 Continuous Memory Allocation

- Divides memory into fixed-sized partitions.
 - Each process occupies one partition.
 - Simple but suffers from **internal fragmentation**.
 - Example:
 - Memory divided into blocks of 10KB.
 - A process requiring 7KB occupies the entire block, wasting 3KB.
-

2.2 Dynamic Memory Allocation

- Allocates memory dynamically at runtime based on process requirements.
 - Reduces fragmentation by utilizing memory efficiently.
 - Example:
 - `malloc()` and `free()` in C dynamically allocate and deallocate memory.
-

3. Memory Allocation Strategies

3.1 First Fit

- Allocates the first available block of memory that is large enough.
 - **Advantages:** Fast.
 - **Disadvantages:** Leads to fragmentation over time.
 - **Example:**
 - Blocks: [10KB, 20KB, 30KB]
 - Process requiring 15KB gets allocated to the 20KB block.
-

3.2 Best Fit

- Allocates the smallest available block that fits the process size.
 - **Advantages:** Reduces wastage.
 - **Disadvantages:** Slower.
 - **Example:**
 - Blocks: [10KB, 20KB, 30KB]
 - Process requiring 15KB gets allocated to the 20KB block.
-

3.3 Worst Fit

- Allocates the largest available block.
 - **Advantages:** May create larger remaining blocks.
 - **Disadvantages:** Wastes large memory chunks.
 - **Example:**
 - Blocks: [10KB, 20KB, 30KB]
 - Process requiring 15KB gets allocated to the 30KB block.
-

4. Compaction

- Combines free memory blocks into a single large block to reduce fragmentation.
 - Moves allocated memory to one end of memory space.
 - Example:
 - Memory layout: [P1, Free(10KB), P2, Free(20KB)]
 - After compaction: [P1, P2, Free(30KB)].
-

5. Fragmentation

5.1 Internal Fragmentation

- Unused memory within allocated blocks.
 - Example:
 - A process requiring 25KB occupies a 30KB block, wasting 5KB.
-

5.2 External Fragmentation

- Free memory exists but is not contiguous.
 - Example:
 - Memory layout: [10KB Free, P1(20KB), 15KB Free].
 - A 25KB process cannot be allocated due to non-contiguous space.
-

6. Segmentation

6.1 What is Segmentation?

- Divides a process into segments (logical divisions like code, data, stack).
 - Each segment is stored in a different memory location.
-

6.2 Hardware Requirements for Segmentation

1. **Base Register:** Stores the starting address of a segment.
 2. **Limit Register:** Stores the size of a segment.
-

6.3 Segmentation Table

- Maps segment numbers to their base and limit addresses.
- Example:

Segment	Base Address	Limit
Code	1000	500
Data	1500	300
Stack	2000	200

7. Paging

7.1 What is Paging?

- Divides physical memory into fixed-sized blocks (**frames**) and processes into blocks of the same size (**pages**).
 - Eliminates external fragmentation.
-

7.2 Hardware Requirements for Paging

1. **Page Table:** Maps logical page numbers to physical frame numbers.
 2. **Translation Lookaside Buffer (TLB):**
 - Cache for page table entries.
 - Speeds up address translation.
-

7.3 Paging Table

- Example:

Page Number	Frame Number
0	5
1	3
2	8

8. Concept of Dirty Bit

- A **dirty bit** indicates if a page has been modified in memory but not written back to disk.
 - **Usage:**
 - Ensures efficient use of I/O by writing only modified pages to disk.
-

9. Shared Pages and Reentrant Code

9.1 Shared Pages

- Allows multiple processes to access the same physical page.
 - Example:
 - Shared libraries are loaded into memory once and accessed by all processes.
-

9.2 Reentrant Code

- Code that can be shared safely among multiple processes.
 - Example:
 - Code sections of `printf()` are reentrant and used concurrently.
-

10. Throttling

- Limits the usage of system resources by processes to prevent overloading.
 - Example:
 - Reducing CPU time for background processes during high system load.
-

11. I/O Management

- Coordinates the flow of data between the CPU, memory, and peripherals.

- Responsibilities:
 - Allocating buffers for I/O.
 - Handling interrupts.
 - Managing device drivers.
 - Example:
 - Writing a file involves buffering data, managing write operations, and notifying completion through interrupts.
-

This comprehensive explanation covers all key aspects of memory management, including allocation strategies, fragmentation, segmentation, paging, and I/O management, along with relevant examples.